

Wazuh

Enterprise Security

Operations Playbook

SIEM | XDR | HIDS | Threat Detection

Prepared by Ashish Patil

Wazuh Security Engineering

May 28, 2026



Contents

1	Introduction to Wazuh	4
1.1	What is Wazuh?	4
1.2	Core Security Concepts	4
1.3	Architecture and Data Flow	4
1.4	Event Processing Pipeline	4
2	Enterprise Architecture	5
2.1	Single-Node Deployment	5
2.2	Distributed Deployment (Production)	5
2.3	Cloud Architecture	5
2.4	MSSP Architecture	5
3	Complete Installation Guide	5
3.1	Ubuntu / Debian Installation (Manager)	6
3.2	CentOS / RHEL / Rocky / AlmaLinux Installation	6
3.3	All-in-One Deployment with the Installation Assistant	6
3.4	Docker Deployment (Single Node)	6
3.5	Post-Installation Verification	7
4	Complete Agent Installation	7
4.1	Windows Agent Installation (GUI and Silent)	7
4.2	Linux Agent (DEB-based)	8
4.3	macOS Agent	8
4.4	Agent Registration and Authentication	8
4.5	Agent Group Assignment via Dashboard	8
4.6	Mass Deployment Strategies	9
5	Complete Configuration Guide (ossec.conf)	9
5.1	Global Settings (Manager)	9
5.2	Remote and Auth (Manager)	9
5.3	Syscheck (File Integrity Monitoring)	9
5.4	Active Response	10
5.5	Vulnerability Detector	10
5.6	Agent Labels	10
6	User Management & Password Management	10
6.1	Dashboard Internal Users	10
6.2	Creating a New Read-only User for SOC L1	11
6.3	API Authentication and Token Usage	11
7	Rules & Decoders Engineering	12
7.1	Custom Rule Syntax (XML)	12
7.2	Frequency Rules (Brute Force Detection)	12
7.3	Correlation Rules (Successful Login After Failures)	12
7.4	Decoder Example (Custom Application JSON)	12
7.5	Testing Rules and Decoders	12
7.6	Sigma Rule Conversion	13

8	Sysmon Integration – Deep Dive	13
8.1	Installing Sysmon on Windows	13
8.2	Recommended Sysmon Configurations	13
8.3	Sysmon Event IDs Quick Reference	13
8.4	Writing Wazuh Rules for Sysmon Events	13
9	Active Response – Automated Countermeasures	14
9.1	Linux Firewall Drop Script	14
9.2	Windows Host Isolation (Netsh)	14
9.3	Enabling Active Response in ossec.conf	14
10	Email Integration	14
10.1	Gmail SMTP with App Password	14
10.2	Office 365 SMTP	15
10.3	Email Alert Level Filtering	15
11	Dashboards & Visualizations – Comprehensive Guide	15
11.1	Wazuh App Built-in Dashboards	15
11.2	Creating Custom Visualizations	15
11.3	Building a Custom SOC Overview Dashboard	16
11.4	Agent Management via Dashboard	16
11.5	Email and Alert Configuration in Dashboard	16
12	Integrations with External Tools	16
12.1	VirusTotal Integration	16
12.2	Slack Webhook Alerting	16
12.3	MISP Threat Intelligence	17
13	Threat Detection Engineering	17
13.1	SSH Brute-Force Attack	17
13.2	Ransomware Behaviour Detection	17
13.3	PowerShell Empire / Cobalt Strike	17
14	File Integrity Monitoring (FIM)	17
14.1	Configuration on Linux	17
14.2	Windows Registry Monitoring	17
14.3	Who-data (Audit)	17
15	Vulnerability Detection	18
15.1	Enabling and Configuring	18
15.2	Interpreting Results	18
15.3	Patch Management Integration	18
16	Compliance Monitoring (Security Configuration Assessment)	18
16.1	PCI-DSS Policy	18
16.2	Custom SCA Policies	18
17	SOC Operations	18
17.1	Daily Monitoring Checklist	18
17.2	Incident Triage Workflow	18
18	Incident Response Playbooks	18
18.1	Malware Outbreak Response	19
18.2	Ransomware Early Stage	19

19 Threat Hunting	19
19.1 Hunting for Persistence Mechanisms	19
19.2 Hunting for Living-off-the-Land Binaries (LOLBins)	19
19.3 Hunting for Lateral Movement	19
20 Troubleshooting	19
20.1 Agent “Never Connected”	19
20.2 Indexer Cluster Red Status	19
20.3 Dashboard Not Loading	19
21 Performance Optimization	19
21.1 Manager Tuning	19
21.2 Indexer Heap Settings	20
21.3 Index Management	20
22 Backup & Recovery	20
22.1 Configuration Backup	20
22.2 Index Snapshots	20
22.3 Disaster Recovery Steps	20
23 API Usage – Automation	20
23.1 Token-based Authentication Script	20
23.2 Common API Endpoints	21
24 Security Hardening	21
24.1 Manager Hardening	21
24.2 TLS Hardening	21
24.3 Dashboard Authentication	21
25 Command Reference	21
26 Real Enterprise Use Cases	21
26.1 Banking SOC	21
26.2 Healthcare Provider	21
26.3 MSSP Multi-Tenant	21
27 Professional Tables	22
27.1 Severity Levels	22
27.2 Required Ports	22
28 Professional Code Blocks	22
28.1 XML: Custom Rule with MITRE	22
28.2 Bash: Agent Mass Registration Script	22
29 Professional Notes Boxes	22
30 Final Conclusion	23

Introduction to Wazuh

What is Wazuh?

Wazuh is a free, open-source security monitoring platform that unifies SIEM, XDR, and HIDS capabilities. It helps organisations detect intrusions, monitor file integrity, scan for vulnerabilities, enforce compliance, and respond automatically to threats. Wazuh collects, aggregates, indexes, and analyses security data from endpoints, servers, cloud workloads, and containers. The platform is built around a central **manager** that processes events, an **indexer** (based on OpenSearch) for storage and search, and a **dashboard** for visualisation and management. Lightweight **agents** installed on monitored endpoints feed data back to the manager over an encrypted channel.

INFO

Wazuh is trusted by thousands of enterprises, MSSPs, and government agencies worldwide for its scalability, transparency, and enterprise-grade feature set.

Core Security Concepts

- **SIEM (Security Information and Event Management)** – Aggregates logs, correlates events, and provides real-time alerting and dashboards.
- **XDR (Extended Detection and Response)** – Extends detection beyond logs to endpoints, network, and cloud, with automated response actions.
- **HIDS (Host-based Intrusion Detection System)** – Monitors internal activity of a host: file changes, registry modifications, rootkit detection, and policy violations.
- **File Integrity Monitoring (FIM)** – Detects unauthorised changes to critical files and system configurations.
- **Vulnerability Detection** – Correlates installed software with public CVE databases (Canonical, Red Hat, NVD) and alerts on unpatched vulnerabilities.
- **Compliance Monitoring** – Automated Security Configuration Assessment (SCA) against standards like PCI-DSS, HIPAA, CIS Benchmarks.
- **Active Response** – Automatically executes countermeasures when a threat is detected (e.g., block an IP, disable a user).

Architecture and Data Flow

The Wazuh platform consists of four main components:

1. **Wazuh Manager** – Central analysis engine. It decodes logs, matches rules, generates alerts, and manages agents. The manager also runs the API (TCP/55000) for remote management.
2. **Wazuh Indexer** – A highly scalable search engine (OpenSearch) that stores alerts and raw events, enabling fast querying and aggregation.
3. **Wazuh Dashboard** – Web-based user interface for threat hunting, visualisation, agent administration, and configuration.
4. **Filebeat** (optional in older versions, but in modern Wazuh the manager pushes directly to the indexer) – If used, it ships alerts from the manager's output to the indexer.

Agents communicate with the manager using AES-256 encryption over TCP port 1514 (data) and 1515 (enrolment). The manager's analysis engine processes events through a pipeline of decoders and rules, stores alerts in flat JSON files, and forwards them to the indexer. The dashboard queries the indexer and presents information in real time.

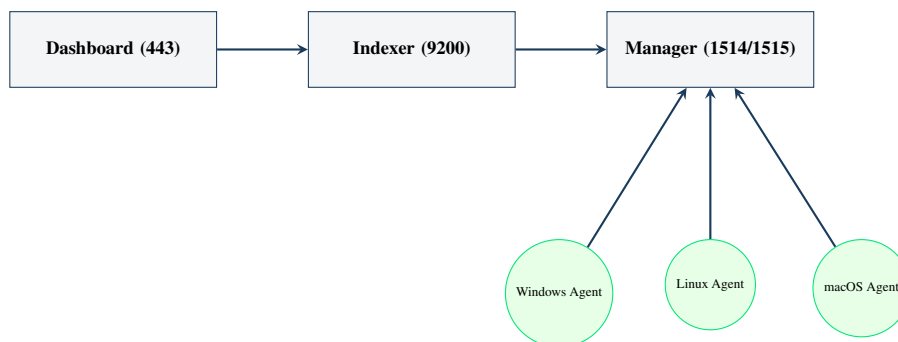


Figure 1: Wazuh communication flow

Event Processing Pipeline

1. **Collection** – Agent collects log events (syslog, Windows Event Log, application logs) or audit events (FIM, registry).
2. **Forwarding** – Encrypted via TCP to manager (1514). For registration, agents use port 1515.

3. **Pre-decoding** – Manager extracts timestamp, hostname, program name.
4. **Decoding** – Relevant decoder extracts structured fields (source IP, user, action).
5. **Rule Matching** – The engine compares decoded fields against rule sets. If a match is found, an alert is generated with severity and MITRE ATT&CK tags.
6. **Alert Enrichment** – GeoIP, threat intelligence, and other enrichments are added.
7. **Output** – Alert is written to `/var/ossec/logs/alerts/alerts.json` and optionally forwarded to the indexer.
8. **Indexing** – The indexer stores the alert and makes it searchable.
9. **Visualisation** – The dashboard queries the indexer and displays alerts, dashboards, and graphs.

TIP

Understanding this pipeline is fundamental for writing custom rules and troubleshooting false positives. Use `wazuh-logtest` to test how a raw log is processed.

Enterprise Architecture

Single-Node Deployment

All components (manager, indexer, dashboard) are installed on the same host. Suitable for labs, PoCs, and small deployments (<100 agents). Quickest to set up using the installation assistant. **Not recommended for production** due to single point of failure and resource contention.

Distributed Deployment (Production)

Separate servers for each component:

- **Manager cluster** – Two or more managers in active-passive or master-worker mode for high availability.
- **Indexer cluster** – Three (or more) nodes for quorum and fault tolerance. Each node stores shards of the data.
- **Dashboard** – Standalone or co-located on a lightweight VM.

Agents can be configured with multiple manager addresses for failover, or a load balancer (e.g., HAProxy) can sit in front of the managers.

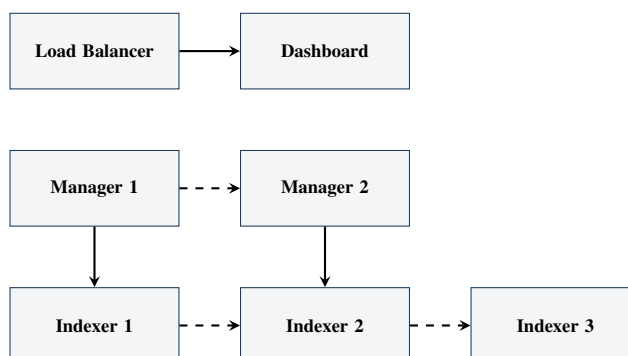


Figure 2: Distributed Wazuh deployment with HA

Cloud Architecture

In AWS, deploy managers in an Auto Scaling group with EBS volumes, behind a Network Load Balancer. Indexers can be on EC2 instances with provisioned IOPS SSDs. Use Security Groups to tightly control traffic. Azure equivalents: Load Balancer, Managed Disks, NSGs.

MSSP Architecture

Multi-tenancy can be achieved by using separate agent groups, dedicated rules, and RBAC-per-customer on the dashboard. A single manager can serve many customers if properly segmented, but larger MSSPs often deploy one manager cluster per customer for isolation.

WARNING

Never expose the manager API (port 55000) or the indexer REST API (9200) directly to the internet. Always use a VPN, reverse proxy with strong authentication, or internal subnets.

Complete Installation Guide

This section provides step-by-step installation instructions for various Linux distributions, covering both all-in-one and distributed setups.

Ubuntu / Debian Installation (Manager)

```
1 # Import GPG key
2 curl -s https://packages.wazuh.com/key/GPG-KEY-WAZUH | sudo gpg --dearmor -o /usr/share/keyrings/
   ↳ wazuh.gpg
3 # Add repository
4 echo "deb [signed-by=/usr/share/keyrings/wazuh.gpg] https://packages.wazuh.com/4.x/apt/ stable
   ↳ main" | sudo tee /etc/apt/sources.list.d/wazuh.list
5 # Update package index
6 sudo apt update
7 # Install Wazuh manager
8 sudo apt install wazuh-manager -y
9 # Enable and start service
10 sudo systemctl daemon-reload
11 sudo systemctl enable wazuh-manager --now
```

Listing 1: Add Wazuh repository and install manager on Ubuntu 22.04

CentOS / RHEL / Rocky / AlmaLinux Installation

```
1 # Import GPG key
2 sudo rpm --import https://packages.wazuh.com/key/GPG-KEY-WAZUH
3 # Create repo file
4 cat <<EOF | sudo tee /etc/yum.repos.d/wazuh.repo
5 [wazuh]
6 gpgcheck=1
7 gpgkey=https://packages.wazuh.com/key/GPG-KEY-WAZUH
8 enabled=1
9 name=Wazuh repository
10 baseurl=https://packages.wazuh.com/4.x/yum/
11 protect=1
12 EOF
13 # Install manager
14 sudo yum install wazuh-manager -y
15 # Enable and start
16 sudo systemctl enable wazuh-manager --now
```

Listing 2: Install Wazuh manager on RHEL 8/9

All-in-One Deployment with the Installation Assistant

The assistant automates the full stack setup. Run the following commands on a fresh Ubuntu 22.04 server.

```
1 # Download the assistant
2 curl -sO https://packages.wazuh.com/4.7/wazuh-install.sh
3 # Generate configuration files and certificates
4 sudo bash wazuh-install.sh --generate-config-files
5 # Install indexer, manager, and dashboard in one go
6 sudo bash wazuh-install.sh --wazuh-indexer node-1
7 sudo bash wazuh-install.sh --start-cluster
8 # After completion, the assistant will output the admin password for the dashboard
```

Listing 3: Wazuh all-in-one installation assistant

INFO

The assistant stores generated passwords and certificates in `/usr/share/wazuh-install-files/`. Keep this directory safe.

Docker Deployment (Single Node)

Create a `docker-compose.yml` file with the following content and run `docker-compose up -d`.

```
1 version: '3.7'
2 services:
```

```
3  wazuh.manager:
4      image: wazuh/wazuh-manager:4.7.2
5      hostname: wazuh.manager
6      restart: always
7      ports:
8          - "1514:1514"
9          - "1515:1515"
10         - "55000:55000"
11      environment:
12          - INDEXER_URL=https://wazuh.indexer:9200
13          - INDEXER_USERNAME=admin
14          - INDEXER_PASSWORD=SecretPassword
15  wazuh.indexer:
16      image: wazuh/wazuh-indexer:4.7.2
17      hostname: wazuh.indexer
18      restart: always
19      ports:
20          - "9200:9200"
21      environment:
22          - "OPENSEARCH_JAVA_OPTS=-Xms1g -Xmx1g"
23  wazuh.dashboard:
24      image: wazuh/wazuh-dashboard:4.7.2
25      hostname: wazuh.dashboard
26      restart: always
27      ports:
28          - "443:5601"
29      environment:
30          - INDEXER_URL=https://wazuh.indexer:9200
31          - INDEXER_USERNAME=admin
32          - INDEXER_PASSWORD=SecretPassword
```

Listing 4: docker-compose.yml for Wazuh single-node

Post-Installation Verification

```
1  # Check manager status
2  sudo systemctl status wazuh-manager
3  # Check that ports are listening
4  ss -tlnp | grep -E '1514|1515|55000'
5  # Verify agent registration daemon (authd)
6  grep "authd" /var/ossec/logs/ossec.log | tail -5
7  # Test API
8  curl -k -u wazuh:wazuh https://localhost:55000/
```

Listing 5: Verify services

Complete Agent Installation

Windows Agent Installation (GUI and Silent)

Download the MSI installer from <https://packages.wazuh.com/4.x/windows/>.

GUI Installation: Run the MSI, provide the manager's IP address and, optionally, an enrolment password and group.

Silent Installation via PowerShell:

```
1  $manager = "10.0.0.10"
2  $group = "windows-servers"
3  Invoke-WebRequest -Uri "https://packages.wazuh.com/4.x/windows/wazuh-agent-4.7.2-1.msi" -OutFile
   ↪ wazuh-agent.msi
4  Start-Process msixec.exe -ArgumentList "/i wazuh-agent.msi /q WAZUH_MANAGER=`"$manager`"
   ↪ WAZUH_REGISTRATION_SERVER=`"$manager`" WAZUH_AGENT_GROUP=`"$group`" -Wait
```

Listing 6: Silent install Windows agent

After installation, the agent service should start automatically. Verify in Services (services.msc) that “Wazuh Agent” is running.

Linux Agent (DEB-based)

```
1 sudo apt install wazuh-agent
2 # Edit the configuration to set the manager address
3 sudo sed -i 's/MANAGER_IP/10.0.0.10/' /var/ossec/etc/ossec.conf
4 # Start the agent
5 sudo systemctl start wazuh-agent
6 # Enable at boot
7 sudo systemctl enable wazuh-agent
```

Listing 7: Install Linux agent (Ubuntu/Debian)

macOS Agent

Download the PKG from the Wazuh website, install it, then edit `/Library/Ossec/ossec.conf` to set the manager IP. Start with `sudo /Library/Ossec/bin/wazuh-control start`.

Agent Registration and Authentication

Agents must be registered on the manager. Two methods:

Manual Registration:

```
1 # Add agent
2 sudo /var/ossec/bin/manage_agents -a <agent-ip> -n <agent-name>
3 # Extract key for agent ID 001
4 sudo /var/ossec/bin/manage_agents -e 001
5 # Copy the key and on the agent run:
6 sudo /var/ossec/bin/manage_agents -i <key>
```

Listing 8: Manual agent registration from manager

Auto-enrolment: Configure the manager’s authd service:

```
1 <auth>
2   <disabled>no</disabled>
3   <port>1515</port>
4   <use_password>yes</use_password>
5   <password>SuperSecret</password>
6 </auth>
```

Listing 9: `/var/ossec/etc/ossec.conf` `<auth>` block

On the agent, set the enrolment password in `ossec.conf`:

```
1 <client>
2   <server>
3     <address>10.0.0.10</address>
4     <port>1514</port>
5     <protocol>tcp</protocol>
6   </server>
7   <enrollment>
8     <enabled>yes</enabled>
9     <password>SuperSecret</password>
10    <agent_name>web-server-01</agent_name>
11    <group>linux-web</group>
12  </enrollment>
13 </client>
```

Agent Group Assignment via Dashboard

Once agents are registered, you can manage groups from the Wazuh Dashboard. Navigate to **Management** → **Groups**. Create a new group (e.g., `linux-web`) and assign agents to it. Groups allow you to push specific configuration files (e.g., `agent.conf`) to all members.

INFO

Use groups to separate configuration profiles: different FIM directories, different log sources, or active response scripts per operating system or function.

Mass Deployment Strategies

- **Ansible** – Write a playbook that installs the agent package and copies a pre-configured `ossec.conf` with the correct manager address and enrolment password.
- **GPO / SCCM** – Deploy the MSI via Group Policy Software Installation or SCCM with the appropriate command-line parameters.
- **Intune** – Package the MSI as a Win32 app and deploy via Microsoft Intune.

Complete Configuration Guide (ossec.conf)

The main configuration file is `/var/ossec/etc/ossec.conf` on both manager and agents. It is XML-based and divided into several blocks. Below are the most important sections with enterprise recommendations.

Global Settings (Manager)

```

1 <ossec_config>
2   <global>
3     <email_notification>yes</email_notification>
4     <smtp_server>smtp.gmail.com</smtp_server>
5     <email_from>wazuh@company.com</email_from>
6     <email_to>soc-alerts@company.com</email_to>
7     <smtp_username>wazuh@gmail.com</smtp_username>
8     <smtp_password>your-app-password</smtp_password>
9     <email_maxperhour>100</email_maxperhour>
10    <alerts_log>yes</alerts_log>
11    <logall>no</logall>
12    <memory_size>1024</memory_size>
13  </global>
14 </ossec_config>

```

Listing 10: Manager global block

Remote and Auth (Manager)

Controls how agents connect.

```

1 <remote>
2   <connection>secure</connection>
3   <port>1514</port>
4   <protocol>tcp</protocol>
5   <queue_size>131072</queue_size>
6 </remote>
7 <auth>
8   <disabled>no</disabled>
9   <port>1515</port>
10  <use_password>yes</use_password>
11  <password>MySecureEnrolmentPass</password>
12 </auth>

```

Listing 11: Remote and auth configuration

Syscheck (File Integrity Monitoring)

```

1 <syscheck>
2   <disabled>no</disabled>
3   <frequency>43200</frequency>
4   <directories check_all="yes" realtime="yes" report_changes="yes">/etc/shadow</directories>
5   <directories check_all="yes">/var/www/html</directories>
6   <ignore>/etc/mtab</ignore>

```

```
7 <windows_registry arch="both">HKEY_LOCAL_MACHINE\Software\Microsoft\Windows\CurrentVersion\Run</  
  ↪ windows_registry>  
8 </syscheck>
```

Listing 12: FIM configuration for critical directories

Active Response

Define commands and attach them to rules.

```
1 <command>  
2 <name>firewall-drop</name>  
3 <executable>firewall-drop.sh</executable>  
4 <expect>srcip</expect>  
5 <timeout_allowed>yes</timeout_allowed>  
6 </command>  
7 <active-response>  
8 <command>firewall-drop</command>  
9 <location>local</location>  
10 <level>10</level>  
11 <timeout>600</timeout>  
12 </active-response>
```

Listing 13: Active response for IP blocking

Vulnerability Detector

```
1 <vulnerability-detector>  
2 <enabled>yes</enabled>  
3 <interval>1h</interval>  
4 <provider name="canonical">  
5 <enabled>yes</enabled>  
6 <os>trusty,xenial,bionic,focal,jammy</os>  
7 </provider>  
8 <provider name="redhat">  
9 <enabled>yes</enabled>  
10 <os>7,8,9</os>  
11 </provider>  
12 <provider name="nvd">  
13 <enabled>yes</enabled>  
14 <update_interval>1h</update_interval>  
15 </provider>  
16 </vulnerability-detector>
```

Listing 14: Vulnerability detector configuration

Agent Labels

Labels allow you to add metadata to agents (e.g., environment, department).

```
1 <labels>  
2 <label key="environment">production</label>  
3 <label key="department">finance</label>  
4 </labels>
```

Listing 15: Agent labels in ossec.conf

User Management & Password Management

Dashboard Internal Users

The Wazuh dashboard uses an internal user database for authentication. Default admin credentials are printed during installation. All password management is done using the `wazuh-passwords-tool.sh` script located under

```
/usr/share/wazuh-dashboard/plugins/wazuh/security/.
```

Reset all passwords to random values:

```
1 sudo /usr/share/wazuh-dashboard/plugins/wazuh/security/wazuh-passwords-tool.sh -a
```

Listing 16: Reset all passwords**Change a single user's password:**

```
1 sudo /usr/share/wazuh-dashboard/plugins/wazuh/security/wazuh-passwords-tool.sh -u kibanaserver -p  
   ↪ NewPassword123
```

Listing 17: Change password for user 'kibanaserver'

Creating a New Read-only User for SOC L1

Edit the internal users file:

```
1 ---  
2 # This file stores the users, passwords, and roles.  
3 soc_l1:  
4   hash: ""  
5   reserved: false  
6   description: "SOC Analyst Level 1  read only"
```

Listing 18: /usr/share/wazuh-dashboard/plugins/wazuh/security/internal_users.yml

Then run the password tool to set the hash:

```
1 sudo /usr/share/wazuh-dashboard/plugins/wazuh/security/wazuh-passwords-tool.sh -u soc_l1 -p  
   ↪ Analyst123!
```

Now map the user to a role in roles_mapping.yml:

```
1 readonly:  
2   users:  
3     - soc_l1  
4     - auditor
```

Listing 19: roles_mapping.yml snippet

Restart the dashboard service:

```
1 sudo systemctl restart wazuh-dashboard
```

API Authentication and Token Usage

The manager API (port 55000) uses basic authentication or JWT tokens. Obtain a token:

```
1 curl -k -X POST https://<manager-ip>:55000/security/user/authenticate \  
2   -H "Content-Type: application/json" \  
3   -d '{"username":"wazuh","password":"wazuh"}'
```

Listing 20: Obtain a JWT token

The response contains a token. Use it for subsequent requests:

```
1 curl -k -X GET "https://<manager-ip>:55000/agents?pretty" \  
2   -H "Authorization: Bearer <token>"
```

TIP

For automation, store the token in a variable and refresh it periodically. The default token validity is 900 seconds (can be changed in API configuration).

Rules & Decoders Engineering

Custom Rule Syntax (XML)

Rules are stored in `/var/ossec/etc/rules/`. For custom rules, always use `local_rules.xml` to prevent them from being overwritten during upgrades.

Basic rule structure:

```
1 <group name="syslog,sshd,">
2   <rule id="100001" level="10">
3     <decoded_as>sshd</decoded_as>
4     <match>Failed password for invalid user</match>
5     <description>SSH brute force attempt invalid user</description>
6     <mitre>
7       <id>T1110</id>
8     </mitre>
9   </rule>
10 </group>
```

Listing 21: Example custom rule for SSH invalid user

Frequency Rules (Brute Force Detection)

Alert when the same event occurs multiple times from the same source within a time window.

```
1 <rule id="100002" level="12" frequency="6" timeframe="30" ignore="60">
2   <if_matched_sid>100001</if_matched_sid>
3   <same_source_ip />
4   <description>Multiple SSH brute force attempts from same IP</description>
5 </rule>
```

Listing 22: Frequency rule for SSH brute force

Correlation Rules (Successful Login After Failures)

Detect a successful login (rule 5710) preceded by failed authentications within 300 seconds from the same IP.

```
1 <rule id="100003" level="15">
2   <if_sid>5710</if_sid>
3   <if_matched_group>authentication_failed</if_matched_group>
4   <same_source_ip />
5   <timeframe>300</timeframe>
6   <description>Successful login after multiple authentication failures</description>
7 </rule>
```

Decoder Example (Custom Application JSON)

Suppose your application writes JSON logs to `/var/log/myapp.log`. Create a decoder:

```
1 <decoder name="myapp">
2   <prematch>^\{"timestamp":</prematch>
3   <plugin_decoder>JSON_Decoder</plugin_decoder>
4 </decoder>
5 <decoder name="myapp_fields">
6   <parent>myapp</parent>
7   <regex>"user": "(\\w+)", \\s*"action": "(\\w+)"</regex>
8   <order>user,action</order>
9 </decoder>
```

Listing 23: Custom JSON decoder

Testing Rules and Decoders

Use `wazuh-logtest` to test your rules:

```
1 sudo /var/ossec/bin/wazuh-logtest
```

Paste a sample log line and it will output the decoded fields and triggered rules. This is essential before deploying any new rule.

Sigma Rule Conversion

Sigma rules can be converted to Wazuh format using `sigmac`:

```
1 sigmac -t wazuh sigma_rule.yml -o local_rules.xml
```

Listing 24: Convert Sigma rule to Wazuh XML

INFO

Always test converted Sigma rules, as they may require tuning for your environment (e.g., field names may differ).

Sysmon Integration – Deep Dive

Installing Sysmon on Windows

Download Sysmon from Microsoft Sysinternals, then install with a configuration file:

```
1 Invoke-WebRequest -Uri https://live.sysinternals.com/Sysmon64.exe -OutFile Sysmon64.exe
2 .\Sysmon64.exe -accepteula -i sysmonconfig-export.xml
```

Listing 25: Install Sysmon with a configuration

Recommended Sysmon Configurations

- **SwiftOnSecurity** – A well-balanced configuration focusing on high-signal events, suitable for most organisations.
- **Olaf Hartong** – A modular, granular configuration that maps events to MITRE ATT&CK techniques; ideal for advanced detection engineering.

Sysmon Event IDs Quick Reference

Event ID	Description
1	Process creation (command line, hashes, parent process)
3	Network connection (source/dest IP, port)
7	Image loaded (DLL)
8	CreateRemoteThread (process injection)
10	ProcessAccess (credential dumping, e.g. LSASS access)
11	FileCreate (ransomware, malware dropping files)
12,13,14	Registry events (key/value create, delete, modify)
17	Pipe created (C2 communication)
18	Pipe connected
22	DNS query (Windows 10+, very useful for C2 detection)

Writing Wazuh Rules for Sysmon Events

The Wazuh agent collects Sysmon events from the Windows Event Log channel `Microsoft-Windows-Sysmon/Operational`. Rules use the `sysmon.event_id` field.

Example: Detect LSASS memory access (credential dumping)

```
1 <rule id="100100" level="12">
2   <decoded_as>sysmon</decoded_as>
3   <field name="sysmon.event_id">10</field>
4   <field name="sysmon.granted_access">0x1010</field>
5   <field name="sysmon.target_image" negate="yes">C:\Windows\system32\csrss.exe</field>
6   <description>Potential LSASS credential dumping (Sysmon Event 10)</description>
7   <mitre><id>T1003.001</id></mitre>
8 </rule>
```

Listing 26: Rule for Sysmon Event 10 – LSASS access**Example: Suspicious PowerShell execution (encoded command)**

```

1 <rule id="100200" level="10">
2   <decoded_as>sysmon</decoded_as>
3   <field name="sysmon.event_id">1</field>
4   <field name="sysmon.image">*\\powershell.exe</field>
5   <field name="sysmon.command_line">.*-enc\s+\S+</field>
6   <description>Suspicious encoded PowerShell command</description>
7 </rule>

```

Active Response – Automated Countermeasures

Linux Firewall Drop Script

Create a script at `/var/ossec/active-response/bin/firewall-drop.sh`:

```

1 #!/bin/bash
2 # Add offending IP to iptables DROP list
3 iptables -I INPUT -s $1 -j DROP
4 # Log the action
5 echo "$(date) Blocked IP $1" >> /var/ossec/logs/active-responses.log

```

Listing 27: firewall-drop.sh

Make it executable: `sudo chmod +x /var/ossec/active-response/bin/firewall-drop.sh`

Windows Host Isolation (Netsh)

Create a batch file `block.bat` in the active-response bin directory:

```

1 netsh advfirewall firewall add rule name="Block %1" dir=in action=block remoteip=%1

```

Listing 28: block.bat – Windows IP block

Enabling Active Response in ossec.conf

```

1 <command>
2   <name>firewall-drop</name>
3   <executable>firewall-drop.sh</executable>
4   <expect>srcip</expect>
5   <timeout_allowed>yes</timeout_allowed>
6 </command>
7 <active-response>
8   <command>firewall-drop</command>
9   <location>local</location>
10  <level>10</level>
11  <timeout>600</timeout>
12 </active-response>

```

Listing 29: Active response command and binding**WARNING**

Active response scripts run with root/administrator privileges. Ensure they are thoroughly tested to avoid self-inflicted denial-of-service.

Email Integration

Gmail SMTP with App Password

1. Enable 2-Step Verification on your Google account.

2. Generate an App Password (select “Mail” and “Other”).
3. Edit manager’s `ossec.conf`:

```
1 <global>
2   <email_notification>yes</email_notification>
3   <smtp_server>smtp.gmail.com</smtp_server>
4   <email_from>wazuh-alerts@gmail.com</email_from>
5   <email_to>soc@company.com</email_to>
6   <smtp_username>wazuh-alerts@gmail.com</smtp_username>
7   <smtp_password>your-16-char-app-password</smtp_password>
8   <email_maxperhour>100</email_maxperhour>
9   <email_alert_level>10</email_alert_level>
10 </global>
```

Listing 30: Gmail SMTP configuration

Restart the manager: `sudo systemctl restart wazuh-manager`.

Office 365 SMTP

Use `smtp.office365.com` as server, port 587, and enable STARTTLS. The password field should contain the user’s Office 365 password (or an app password if MFA is enforced).

Email Alert Level Filtering

Set `<email_alert_level>` to only receive alerts at or above a certain severity. For SOC use, level 10 or higher is recommended to reduce noise.

TIP

Test email delivery by manually triggering a high-level alert, or use the `logger` command to send a test log that matches a rule with level 10+.

Dashboards & Visualizations – Comprehensive Guide

The Wazuh Dashboard provides a rich interface for monitoring, hunting, and managing the platform. This section covers both built-in dashboards and how to create your own visualizations.

Wazuh App Built-in Dashboards

Access via the Wazuh menu:

- **Security Operations** – Alert summary, top rules, agents, and alert evolution over time.
- **Compliance** – SCA check results by policy and agent.
- **Vulnerabilities** – Detected CVEs by severity and affected agents.
- **MITRE ATT&CK** – Matrix showing technique coverage; click to drill down.
- **File Integrity** – File changes and who made them.
- **Agents** – Status, OS, groups, and last keep alive.

Creating Custom Visualizations

You can extend the dashboard using OpenSearch Dashboards visualisations. Steps:

1. Navigate to **OpenSearch Dashboards** → **Visualize**.
2. Click **Create visualization** and select a type (e.g., Vertical Bar, Pie, Data Table).
3. Choose the index pattern `wazuh-alerts-*`.
4. Build your query:
 - **KQL**: `rule.level >= 10 and rule.groups: "sysmon"`
 - **DSL** (under “Options”):

```
1 {
2   "query": {
3     "bool": {
4       "must": [{"term": {"rule.level": 12}}]
5     }
6   },
7   "aggs": {
```



```

8     "top_srcip": {
9         "terms": {"field": "data.srcip", "size": 10}
10    }
11 }
12 }
13

```

Listing 31: DSL query for top source IPs

5. Configure the metrics (e.g., Count, Average of `rule.level`).
6. Define buckets: X-axis = Terms on `data.srcip`, order by count descending.
7. Save the visualization with a descriptive name (e.g., “Top Attacking IPs”).

Building a Custom SOC Overview Dashboard

After creating multiple visualizations, assemble them into a dashboard:

1. Go to **OpenSearch Dashboards** → **Dashboard**.
2. Click **Create new dashboard**, then **Add** existing visualizations.
3. Arrange and resize panels by dragging.
4. Add a time range filter (e.g., “Last 24 hours”).
5. Save the dashboard as “SOC Daily Operations”.

Now you can share it with team members or export via the dashboard management API.

Agent Management via Dashboard

The **Agents** tab allows you to view all registered agents, their status (active, disconnected, never connected), OS, group, and last keep alive. You can:

- Restart an agent remotely.
- Upgrade the agent software.
- View agent configuration and logs.
- Add or remove labels.
- Move agents to groups (right-click on agent → Edit groups).

Group Management: Navigate to **Management** → **Groups**. Create a new group, e.g., “PCI-servers”. Then under the group, you can add configuration files (`agent.conf`) that will be pushed to all members of that group. For example, you could specify a custom log collection or FIM configuration only for those servers.

Email and Alert Configuration in Dashboard

While the SMTP settings are configured in the manager’s `ossec.conf`, the dashboard provides a **Settings** → **Configuration** page that shows the current configuration of each node (manager, agents). You can verify that the email settings are active by checking the manager’s configuration from this interface. Also, the **Management** → **Rules** section allows you to search and tune rules, helping identify which rules are generating the most alerts and potentially need threshold adjustments.

Integrations with External Tools

VirusTotal Integration

Wazuh can query VirusTotal for file hashes detected by FIM or Sysmon.

```

1 <integration>
2   <name>virustotal</name>
3   <api_key>YOUR_VIRUSTOTAL_API_KEY</api_key>
4   <group>syscheck</group>
5   <alert_format>json</alert_format>
6   <hook_url>https://www.virustotal.com/vtapi/v2/file/report</hook_url>
7   <rule_id>554,550</rule_id>
8 </integration>

```

Listing 32: Manager `ossec.conf` – VirusTotal integration

Slack Webhook Alerting

Create a custom integration script (`custom-slack.py`) that reads `alerts.json` and POSTs to a Slack incoming webhook. Place it under `/var/ossec/integrations/` and configure the integration block accordingly. Many community scripts are

available.

MISP Threat Intelligence

Feed MISP events to Wazuh for IOC matching. Requires a script that fetches indicators and converts them to Wazuh CDB lists.

Threat Detection Engineering

SSH Brute-Force Attack

Log sample:

```
1 sshd[12345]: Failed password for invalid user admin from 203.0.113.42 port 2222 ssh2
```

Rule (already built-in 5712 for invalid user, 5710 for root). Tune with frequency rule as shown earlier.

Ransomware Behaviour Detection

- FIM alerts on mass file renames (.encrypted extension).
- Sysmon Event 11 (FileCreate) for known ransomware notes (README.txt, HOW_TO_DECRYPT).
- Windows Event ID 524 (System log) for Volume Shadow Copy deletion.

Custom rule example for VSS deletion:

```
1 <rule id="100300" level="15">
2   <decoded_as>windows</decoded_as>
3   <field name="win.system.eventID">524</field>
4   <description>Volume Shadow Copy deleted possible ransomware</description>
5   <mitre><id>T1490</id></mitre>
6 </rule>
```

PowerShell Empire / Cobalt Strike

Detect encoded commands and suspicious network connections (C2).

- Sysmon Event 1 with powershell.exe -enc ...
- Network connection (Sysmon Event 3) to unusual foreign IP on high ports.

File Integrity Monitoring (FIM)

Configuration on Linux

```
1 <syscheck>
2   <directories check_all="yes" realtime="yes" report_changes="yes">/etc/passwd,/etc/shadow,/etc/
   ↪ sudoers</directories>
3   <directories check_all="yes" report_changes="yes">/home/*/.ssh/authorized_keys</directories>
4   <frequency>3600</frequency>
5 </syscheck>
```

Windows Registry Monitoring

```
1 <syscheck>
2   <windows_registry arch="both">HKEY_LOCAL_MACHINE\SYSTEM\CurrentControlSet\Services</
   ↪ windows_registry>
3   <windows_registry>HKEY_CURRENT_USER\Software\Microsoft\Windows\CurrentVersion\Run</
   ↪ windows_registry>
4 </syscheck>
```

Who-data (Audit)

To see which user made a change, enable the audit option and install auditd on Linux. On Windows, the agent automatically retrieves the user if auditing is enabled.

Vulnerability Detection

Enabling and Configuring

```
1 <vulnerability-detector>
2   <enabled>yes</enabled>
3   <interval>1h</interval>
4   <provider name="canonical"><enabled>yes</enabled><os>bionic,focal,jammy</os></provider>
5   <provider name="redhat"><enabled>yes</enabled><os>8,9</os></provider>
6   <provider name="nvd"><enabled>yes</enabled><update_interval>1h</update_interval></provider>
7 </vulnerability-detector>
```

Interpreting Results

In the Wazuh Dashboard, the “Vulnerabilities” tab lists CVEs by severity (Critical, High, Medium, Low). Each entry shows the affected package, version, and CVSS score. You can filter by agent and export reports.

Patch Management Integration

Wazuh doesn’t patch automatically, but you can use the vulnerability data to prioritise patching. Integrate with WSUS (Windows), Landscape (Ubuntu), or Red Hat Satellite.

Compliance Monitoring (Security Configuration Assessment)

PCI-DSS Policy

Wazuh ships with an SCA policy for PCI-DSS 3.2.1. Enable it in the agent’s `ossec.conf`:

```
1 <wodle name="sca">
2   <enabled>yes</enabled>
3   <scan_on_start>yes</scan_on_start>
4   <interval>12h</interval>
5   <policies>
6     <policy>cis_pci_dss_3.2.1.yml</policy>
7   </policies>
8 </wodle>
```

Results are visible under the Compliance tab.

Custom SCA Policies

Write your own YAML policy file (e.g., `company_baseline.yml`) and place it in `/var/ossec/etc/sca/` on the manager. The dashboard allows you to view pass/fail per check and export compliance reports.

SOC Operations

Daily Monitoring Checklist

1. Check agent status: ensure no critical systems are “Disconnected” or “Never connected”.
2. Review high-severity alerts (level ≥ 12) from the last 24 hours.
3. Inspect the vulnerability dashboard for new critical CVEs.
4. Verify FIM changes on sensitive files (e.g., `/etc/shadow`).
5. Check indexer health (cluster status green, no unassigned shards).
6. Review email alert queue; ensure no alert storms are occurring.

Incident Triage Workflow

1. **L1 Analyst** – Acknowledges alert, validates severity. If false positive, tune rule or add exception. If true positive, escalate to L2.
2. **L2 Analyst** – Performs deeper investigation using dashboards and raw logs. May trigger active response.
3. **L3/Incident Response** – Full incident response, containment, eradication, and post-mortem.

Incident Response Playbooks

Malware Outbreak Response

1. Wazuh alert indicates malware (hash match, suspicious process).
2. Isolate host: Use active response script that blocks all network traffic except to the management console.
3. Trigger a full Microsoft Defender scan via Wazuh active response (PowerShell script: `Start-MpScan -ScanType FullScan`).
4. Collect forensic artifacts (prefetch, event logs) from the agent.
5. If confirmed, re-image the host.

Ransomware Early Stage

1. FIM alerts on massive file changes or VSS deletion.
2. Immediately isolate affected servers (Active Response – drop all inbound/outbound except management).
3. Block known C2 IPs at perimeter firewall.
4. Notify SOC manager and initiate IR plan.
5. Restore from offline backup after cleaning.

Threat Hunting

Hunting for Persistence Mechanisms

Query: `sysmon.event_id: 13 AND sysmon.target_object: *Run*` to find registry auto-start entries.

Hunting for Living-off-the-Land Binaries (LOLBins)

KQL: `sysmon.event_id: 1 AND (sysmon.image: *mshta.exe OR sysmon.image: *rundll32.exe OR sysmon.image: *certutil.exe)`. Look for unusual parent process or command line arguments.

Hunting for Lateral Movement

Look for Sysmon Event 3 (network connection) where destination port is 445 or 3389 and source is a workstation, not a server.

Troubleshooting

Agent “Never Connected”

1. Check agent's `ossec.conf` for correct manager IP and port.
2. Ensure the manager's firewall allows incoming TCP 1514 from the agent's IP.
3. On the agent, run `telnet <manager-ip> 1514` to test connectivity.
4. Look at agent log: `/var/ossec/logs/ossec.log` for “Unable to connect” errors.
5. Verify the agent key exists and matches: `/var/ossec/bin/manage_agents -l` on manager and `/var/ossec/etc/client.keys` on agent.

Indexer Cluster Red Status

- Check disk space and heap usage on each node.
- Ensure quorum: at least 2 out of 3 master-eligible nodes must be up.
- Look at indexer logs: `/var/log/wazuh-indexer/wazuh-indexer.log`.
- Restart the disconnected nodes.

Dashboard Not Loading

- Check dashboard service: `systemctl status wazuh-dashboard`.
- Verify indexer is reachable: from dashboard host, `curl -k https://<indexer>:9200`.
- Check dashboard logs: `journalctl -u wazuh-dashboard -n 50`.
- Ensure TLS certificates are valid and not expired.

Performance Optimization

Manager Tuning

In `ossec.conf`, under `<global>`, increase `<memory_size>` to 2048 or higher for large deployments. Adjust `<queue_size>` in the `<remote>` block to prevent event drops.

Indexer Heap Settings

Edit `/etc/wazuh-indexer/jvm.options`:

```
1 -Xms8g
2 -Xmx8g
```

Set the heap to 50% of available RAM, but no more than 32GB.

Index Management

Apply a rollover policy to keep shard sizes manageable. Use the Index State Management (ISM) plugin to delete or archive old indices after 30 days.

Backup & Recovery

Configuration Backup

Use a script to tar manager and dashboard configs:

```
1 #!/bin/bash
2 tar -czf /backup/wazuh-config-$(date +%F).tar.gz \
3   /var/ossec/etc/ /etc/filebeat/ /etc/wazuh-dashboard/ /etc/wazuh-indexer/
4 # Optionally push to S3 or NFS
```

Listing 33: config-backup.sh

Index Snapshots

Create an ISM policy that takes daily snapshots to an S3 repository. In case of disaster, restore from snapshot.

Disaster Recovery Steps

1. Provision a new server with the same Wazuh version.
2. Restore configuration tarball to `/`.
3. Start Wazuh manager.
4. Restore indexer from snapshot.
5. Re-enrol agents if keys were lost.

API Usage – Automation

Token-based Authentication Script

```
1 import requests, json, urllib3
2 urllib3.disable_warnings()
3 base = 'https://manager:55000'
4 # Get token
5 auth_resp = requests.post(f'{base}/security/user/authenticate',
6                           auth=('wazuh', 'wazuh'), verify=False)
7 token = auth_resp.json()['data']['token']
8 headers = {'Authorization': f'Bearer {token}'}
9 # List agents
10 agents = requests.get(f'{base}/agents', headers=headers, verify=False)
11 print(json.dumps(agents.json(), indent=2))
12 # Restart an agent
13 requests.put(f'{base}/agents/001/restart', headers=headers, verify=False)
```

Listing 34: Python API client example

Common API Endpoints

Endpoint	Description
GET /agents	List all agents with status
GET /agents/001	Get details of agent 001
PUT /agents/001/restart	Restart agent 001
GET /alerts?limit=5	Retrieve last 5 alerts
GET /manager/info	Manager version and status

Security Hardening

Manager Hardening

- Run the manager as a non-root user (the service drops privileges, but ensure file permissions are correct).
- Enable SELinux or AppArmor profiles for Wazuh.
- Restrict API access to localhost or management subnet using firewall.

TLS Hardening

Use the `wazuh-cert-tool.sh` to generate certificates with strong keys (2048-bit RSA). Enforce TLS 1.2+ in `ossec.conf` and indexer configuration.

Dashboard Authentication

Place the dashboard behind a reverse proxy (Nginx/Apache) with SAML/OIDC integration and enable 2FA for administrators.

Command Reference

Command	Purpose
<code>/var/ossec/bin/agent_control -l</code>	List all agents and status
<code>/var/ossec/bin/manage_agents</code>	Add, remove, extract agent keys
<code>systemctl restart wazuh-manager</code>	Restart the manager
<code>/var/ossec/bin/wazuh-logtest</code>	Test rules against raw log
<code>curl -k -u wazuh:wazuh https://localhost:55000/</code>	API health check
<code>ss -tlnp grep 1514</code>	Check if manager is listening

Real Enterprise Use Cases

Banking SOC

Wazuh deployed across 5,000 endpoints for continuous compliance (PCI-DSS, SOX). Custom rules monitor SWIFT transaction logs and detect anomalies. Active response isolates compromised ATMs.

Healthcare Provider

HIPAA compliance with FIM on EMR databases, vulnerability scanning on medical IoT devices, and real-time alerting on unauthorised access to patient records.

MSSP Multi-Tenant

Each customer has a dedicated agent group and role-based dashboard access. Centralised manager cluster handles 20,000 agents with custom rules per customer.

Professional Tables

Severity Levels

Level	Classification
0–4	Informational / No security impact
5–7	Low severity, suspicious activity
8–11	High severity, likely attack
12–15	Critical, confirmed compromise

Required Ports

Service	Port	Proto	Purpose
Agent data	1514	TCP	Encrypted event channel
Agent enrolment	1515	TCP	Registration and key exchange
Manager API	55000	TCP	RESTful management interface
Indexer REST	9200	TCP	Data indexing and search
Dashboard	443	TCP	HTTPS web UI

Professional Code Blocks

XML: Custom Rule with MITRE

```

1 <rule id="100400" level="10">
2   <decoded_as>sysmon</decoded_as>
3   <field name="sysmon.event_id">1</field>
4   <field name="sysmon.image">*\\shtasks.exe</field>
5   <field name="sysmon.command_line">*/create*/tn*</field>
6   <description>New scheduled task created via shtasks</description>
7   <mitre><id>T1053.005</id></mitre>
8 </rule>

```

Listing 35: Rule for suspicious scheduled task creation

Bash: Agent Mass Registration Script

```

1 #!/bin/bash
2 # Reads a list of IPs and registers them
3 while read ip name; do
4   /var/ossec/bin/manage_agents -a $ip -n $name
5 done < agents.txt

```

Listing 36: register-agents.sh

Professional Notes Boxes

TIP

Always test new rules in a staging environment using `wazuh-logtest` before promoting to production.

WARNING

Do not use self-signed certificates in production. Generate proper certificates with the `wazuh-cert-tool.sh` and distribute them securely.

!! CRITICAL

Immediately investigate any alert tagged with MITRE T1003 (credential dumping) or T1021 (lateral movement). These are often precursors to full compromise.

Final Conclusion

This playbook has equipped you with the knowledge to build, operate, and mature a Wazuh-based SOC. From architecture planning to rule engineering, dashboard creation, and incident response, every aspect has been covered in detail. The open-source nature of Wazuh, combined with rigorous security practices, delivers enterprise-grade threat detection and response capabilities without the licensing burden.

Recommended Next Steps:

1. Deploy a distributed Wazuh environment in a lab.
2. Tune default rules and build custom ones for your critical applications.
3. Create role-based dashboards for your SOC tiers.
4. Integrate threat intelligence feeds and automate response playbooks.
5. Perform regular backup and disaster recovery tests.

For the latest updates, always refer to the official Wazuh documentation: <https://documentation.wazuh.com>.